

DEVOPS ENGINEERING

From Beginner to Master

The Complete Production-Ready Handbook
Commands, Manifests, Scripts & Real-World Blueprints

Curriculum Scope	Linux • Git • Python & Bash • Docker • Kubernetes • Terraform • Ansible • CI/CD (GitHub Actions / Jenkins) • Prometheus & Grafana • Site Reliability
Target Audience	Aspiring DevOps Engineers, Cloud Engineers, SREs, Systems Administrators, and Developers transitioning to Cloud Infrastructure.
Structure	11 Core Practical Modules with runnable commands, configuration manifests, automated scripts, and an Enterprise Capstone Architecture.
Standard	Modern Tooling Standard (Kubernetes v1.30+, Docker Compose v2, Terraform v1.8+, GitHub Actions, Prometheus).

Module 1: Linux Administration & Shell Essentials

Module 2: Linux Networking, Security & Remote Access

Module 3: Version Control & Git Collaboration

Module 4: Shell Scripting & Python Automation

Module 5: Containerization with Docker

Module 6: Container Orchestration with Kubernetes

Module 7: Infrastructure as Code with Terraform

Module 8: Configuration Management with Ansible

Module 9: CI/CD Pipelines (GitHub Actions & Jenkins)

Module 10: Observability, SRE & Production Debugging

Module 11: Enterprise Capstone Project & Production Checklist

Appendix: 12-Week Structured Master Roadmap

Module 1: Linux Administration & Shell Essentials

Linux is the backbone of modern cloud infrastructure. Mastery begins with confident navigation, permissions, stream processing, and system diagnostics.

1. File System Navigation & Manipulation

<code>pwd</code>	Print current absolute working directory.
<code>ls -lha</code>	List all entries including hidden files with permissions, owner, and human-readable sizes.
<code>mkdir -p /opt/app/{bin,config,logs}</code>	Create deep directory tree structures in a single command.
<code>cp -av /src /dest_backup</code>	Archive copy preserving mode, ownership, timestamps, and symlinks.
<code>mv -u source.tar /storage/</code>	Move file only when source is newer than destination or missing.
<code>rm -rf /tmp/build_cache</code>	Forcefully and recursively remove files and directories.
<code>find /var/log -type f -name '*.log' -mtime +7 -exec rm -f {} \;</code>	Find <code>.log</code> files older than 7 days and delete them automatically.
<code>df -hT</code>	Display total, used, and free disk space across all mounted file systems with types.
<code>du -ah --max-depth=1 /var/log sort -hr</code>	Inspect disk consumption per directory sorted descending.

2. Ownership, User Permissions & Security

<code>chmod 755 deploy.sh</code>	Set <code>rxr-xr-x</code> (Owner: read/write/execute; Group & Others: read/execute).
<code>chmod -R 600 ~/.ssh/id_rsa</code>	Restrict SSH private key permissions strictly to owner read/write.
<code>chown -R www-data:www-data /var/www</code>	Change owner and primary group recursively.
<code>useradd -m -s /bin/bash -G sudo devops</code>	Create user 'devops' with home directory, bash shell, and sudo privileges.
<code>usermod -aG docker devops</code>	Add user to docker group to allow running containers without sudo.
<code>sudo visudo</code>	Safely edit the sudoers configuration file with syntax validation.

3. Text Processing & Data Extraction Pipelines

<code>grep -rn 'ERROR FATAL' /var/log/app/</code>	Search recursively for regex patterns with matching line numbers.
<code>awk '\$9 == 500 {print \$1, \$7, \$9}' access.log</code>	Parse column fields (e.g., filter client IP and URL on HTTP 500 errors).
<code>sed -i 's/ENVIRONMENT=dev/ENVIRONMENT=prod/g' config.env</code>	In-place string replacement across configuration files.
<code>cat access.log cut -d ' ' -f 1 sort uniq -c sort -nr head -n 10</code>	Top 10 highest-frequency client IP addresses hitting the web server.
<code>xargs -I {} -P 4 gzip {}</code>	Execute parallel compression tasks utilizing multiple CPU cores.

4. Process, Services & Performance Monitoring

<code>ps aux grep [n]ode</code>	List all active node processes without matching the grep command itself.
<code>systemctl list-unit-files --type=service --state=running</code>	List all actively running systemd services.
<code>systemctl enable --now nginx</code>	Enable nginx to start on system boot and start it immediately.
<code>journalctl -u nginx -n 100 --no-pager</code>	View last 100 log lines of nginx service without interactive paging.
<code>kill -15 / kill -9</code>	Send SIGTERM (graceful shutdown) or SIGKILL (force termination).
<code>free -h / vmstat 1 5</code>	Display RAM/swap utilization and virtual memory statistics every second.
<code>uptime / iostat -xz 1 3</code>	Check system load average and deep disk I/O latency/utilization.

Module 2: Linux Networking, Security & Remote Access

Infrastructure reliability requires a deep understanding of network connectivity, port binding, firewall rules, and hardened SSH tunnels.

1. Network Diagnostics & Socket Inspection

<code>ip addr show / ip route</code>	Display interface IP configurations and kernel routing tables.
<code>ss -tulnp</code>	List all listening TCP/UDP sockets, bound addresses, and associated process PIDs.
<code>curl -Iv https://api.service.internal</code>	Detailed HTTP request/response verbose diagnostics and SSL certificate verification.
<code>dig +short +trace api.example.com</code>	Trace authoritative DNS resolution from root servers to leaf record.
<code>nc -zv -w 3 10.0.0.15 5432</code>	Test TCP socket connectivity to PostgreSQL database port with 3s timeout.
<code>tcpdump -i eth0 -nn -s0 -w dump.pcap 'port 80 or port 443'</code>	Capture full payload network packets on interface for Wireshark analysis.
<code>traceroute -n -m 30 8.8.8.8</code>	Trace network route hops to target host without reverse DNS lookups.

2. Firewall Configuration (UFW & iptables)

<code>sudo ufw default deny incoming</code>	Set baseline security policy to deny all unsolicited incoming traffic.
<code>sudo ufw default allow outgoing</code>	Allow all outgoing connections initiated from the server.
<code>sudo ufw allow 22/tcp comment 'SSH Access'</code>	Open SSH port with inline administrative documentation comment.
<code>sudo ufw allow from 10.0.0.0/24 to any port 3306 proto tcp</code>	Restrict MySQL database access strictly to private subnet.
<code>sudo ufw enable && sudo ufw status verbose</code>	Enable firewall rules and inspect current active security profile.

3. SSH Hardening & Infrastructure Key Management

<code>ssh-keygen -t ed25519 -C 'devops_deploy'</code>	Generate modern, secure Ed25519 elliptic curve keypair.
<code>ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server_ip</code>	Safely append public key to remote host's ~/.ssh/authorized_keys.
<code>ssh -L 9000:localhost:9000 -N -f user@remote</code>	Establish background local port forwarding tunnel to access remote UI.
<code>ssh -R 8080:localhost:3000 user@public_ip</code>	Remote reverse port forwarding to expose local development service.
<code>rsync -avzP --exclude='.git' ./build/ user@server:/var/www/</code>	Fast, compressed, differential directory synchronization over SSH.

Production Hardened SSH Config (~/.ssh/config):

```
Host prod-bastion
HostName bastion.production.net
User deployer
IdentityFile ~/.ssh/id_ed25519
Port 2222
ServerAliveInterval 30
ServerAliveCountMax 3

Host prod-app-*
User ubuntu
ProxyJump prod-bastion
IdentityFile ~/.ssh/id_ed25519
StrictHostKeyChecking accept-new
```

Module 3: Version Control & Git Collaboration

Git is the foundational ledger for source code, GitOps, and Infrastructure as Code. Precision with branch management and history rewriting is essential.

1. Core Workflow & Staging

<code>git init -b main</code>	Initialize new git repository with default branch 'main'.
<code>git clone --depth 1</code>	Shallow clone fetching only the latest commit to optimize CI pipeline speed.
<code>git status -sb</code>	Display compact, short-format status with branch tracking info.
<code>git add -p</code>	Interactively review and select individual change hunks for staging.
<code>git commit -m 'feat(auth): add jwt token validation'</code>	Create commit following Conventional Commits specification.
<code>git log --oneline --graph --decorate --all</code>	Visual ASCII representation of all branches and commit histories.
<code>git diff --staged</code>	Inspect exact line changes staged for the upcoming commit.

2. Branching, Merging & Rebase Operations

<code>git switch -c feature/payments</code>	Create and switch to a new feature branch in one step.
<code>git fetch --all --prune</code>	Fetch all remote refs and delete local references to deleted remote branches.
<code>git rebase main</code>	Replay current branch commits cleanly on top of updated main branch.
<code>git rebase -i HEAD~4</code>	Interactive rebase to squash, fixup, reword, or drop past 4 commits.
<code>git merge --no-ff feature/payments</code>	Merge feature branch with a dedicated merge commit preserving history.
<code>git rebase --continue / git rebase --abort</code>	Advance or safely cancel ongoing rebase after conflict resolution.

3. Advanced Git Diagnostics & Recovery

<code>git stash push -m 'wip-cache' --include-untracked</code>	Stash working directory modifications including newly created files.
<code>git stash pop</code>	Apply most recent stash and remove it from stash list.
<code>git cherry-pick</code>	Apply a specific commit from another branch into current branch.
<code>git reflog</code>	Display complete chronological log of local HEAD movements (safety net).
<code>git reset --hard HEAD@{2}</code>	Restore branch state to where HEAD was 2 actions ago via reflog.
<code>git revert -m 1</code>	Create a new inverse commit to safely undo a merged pull request.
<code>git bisect start HEAD v1.0.0</code>	Binary search through commit history to automatically pinpoint bug introduction.
<code>git clean -fd</code>	Forcefully remove all untracked files and directories.

Module 4: Shell Scripting & Python Automation

Automation eliminates manual toil. Production automation scripts must be idempotent, defensively written, and resilient to transient errors.

1. Production-Grade Bash Scripting Standards

Always enforce strict error handling at the beginning of shell scripts:

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'

readonly TIMESTAMP="$(date +%Y%m%d_%H%M%S)"
readonly BACKUP_ROOT="/var/backups/app"
readonly TARGET_ARCHIVE="${BACKUP_ROOT}/backup_${TIMESTAMP}.tar.gz"
readonly RETENTION_DAYS=14

log() { echo "[$(date +%Y-%m-%dT%H:%M:%S%z)] [${1}] ${2}"; }

cleanup() {
  local exit_code=$?
  if [ ${exit_code} -ne 0 ]; then
    log "ERROR" "Backup script failed with exit code: ${exit_code}"
  fi
  exit ${exit_code}
}
trap cleanup EXIT INT TERM

main() {
  log "INFO" "Starting automated backup procedure..."
  mkdir -p "${BACKUP_ROOT}"
  log "INFO" "Creating archive: ${TARGET_ARCHIVE}"
  tar -czf "${TARGET_ARCHIVE}" -C /var/www/app config data
  log "INFO" "Pruning archives older than ${RETENTION_DAYS} days..."
  find "${BACKUP_ROOT}" -type f -name "backup_*.tar.gz" -mtime "+${RETENTION_DAYS}" -delete
  log "INFO" "Backup completed successfully."
}
main "$@"
```

2. Python for DevOps & Infrastructure Monitoring

```
import subprocess
import time
import requests

SERVICES = ["nginx", "docker", "redis-server"]

def check_service(service: str) -> bool:
    cmd = ["systemctl", "is-active", "--quiet", service]
    return subprocess.run(cmd, check=False).returncode == 0

def restart_service(service: str) -> bool:
    print(f"Attempting restart for {service}...")
    return subprocess.run(["sudo", "systemctl", "restart", service], check=False).returncode == 0

def monitor_infrastructure():
    for svc in SERVICES:
        if not check_service(svc):
            print(f"[ALERT] Service {svc} is DOWN! Triggering restart...")
            if restart_service(svc):
                time.sleep(2)
            if check_service(svc):
                print(f"[INFO] Service {svc} successfully recovered.")
            else:
                print(f"[CRITICAL] Auto-restart failed for {svc}!")

if __name__ == "__main__":
    monitor_infrastructure()
```

Module 5: Containerization with Docker

Containers package code, runtime, system tools, and dependencies together. Docker guarantees software runs identically across local development, staging, and production clusters.

1. Essential Docker CLI Commands

<code>docker build -t myapp:1.0.0 -f Dockerfile .</code>	Build container image from Dockerfile with release tag.
<code>docker run -d --name web -p 8080:80 --restart=always -v app_data:/data nginx:alpine</code>	Run container detached, mapped port 8080->80, auto-restart, named volume.
<code>docker ps -a --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'</code>	Formatted tabular output of all container statuses.
<code>docker exec -it web /bin/sh</code>	Attach interactive terminal session inside running container.
<code>docker logs -f --tail=100 web</code>	Follow real-time standard output / standard error streams.
<code>docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' web</code>	Extract container internal IP address via Go template.
<code>docker network create --driver bridge internal_net</code>	Create dedicated isolated bridge network for inter-container communication.
<code>docker volume create pg_data</code>	Create managed persistent volume decoupled from container lifecycle.
<code>docker system prune -af --volumes</code>	Deep cleanup removing all stopped containers, unused networks, images, and volumes.

2. Optimized Multi-Stage Dockerfile

```
# Stage 1: Build & Dependencies
FROM node:20-alpine AS builder
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

# Stage 2: Minimal Production Runtime
FROM nginx:1.27-alpine-slim
LABEL maintainer="devops@company.com"
RUN rm -rf /usr/share/nginx/html/*
COPY --from=builder /usr/src/app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
USER nginx
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget --quiet --tries=1 --spider http://localhost:8080/health || exit 1
CMD ["nginx", "-g", "daemon off;"]
```

3. Production Docker Compose (docker-compose.yml)

```
services:
  app:
    build: .
    ports: ["8080:8080"]
    environment:
      - DATABASE_URL=postgres://db_user:secret123@postgres:5432/app_db
    depends_on:
      postgres:
        condition: service_healthy
    networks: [backend_network]

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: db_user
      POSTGRES_PASSWORD: secret123
      POSTGRES_DB: app_db
    volumes: [postgres_data:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U db_user -d app_db"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks: [backend_network]

volumes:
  postgres_data:
networks:
  backend_network:
    driver: bridge
```

Module 6: Container Orchestration with Kubernetes

Kubernetes automates deployment, scaling, self-healing, and networking of containerized application fleets.

1. Essential kubectl Commands

<code>kubectl get nodes -o wide</code>	List all cluster worker/control-plane nodes, status, versions, and IP addresses.
<code>kubectl get pods -A --field-selector=status.phase!=Running</code>	Find all failing/unhealthy pods across all namespaces.
<code>kubectl apply -f manifest.yaml</code>	Declaratively apply Kubernetes configuration manifest to active cluster.
<code>kubectl describe pod -n production</code>	Inspect pod lifecycle events, volume mounts, resource limits, and error reasons.
<code>kubectl logs -f deployment/api-server -c app -n production</code>	Stream logs from a specific container inside a deployment.
<code>kubectl exec -it -n production -- /bin/sh</code>	Execute interactive shell inside target pod container.
<code>kubectl scale deployment/api-server --replicas=5 -n prod</code>	Scale deployment replica count imperatively.
<code>kubectl rollout restart deployment/api-server -n prod</code>	Trigger rolling restart of pods without downtime.
<code>kubectl rollout undo deployment/api-server --to-revision=2 -n prod</code>	Instantly rollback deployment to specified revision.
<code>kubectl port-forward svc/api-service 8080:80 -n prod</code>	Forward local workstation port directly to in-cluster service.
<code>kubectl top nodes && kubectl top pods -A</code>	View live CPU & memory consumption per node and per pod.

2. Production Kubernetes Deployment & Service Manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-service
  namespace: production
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
  selector:
    matchLabels:
      app: api-service
  template:
    metadata:
      labels:
        app: api-service
    spec:
      containers:
        - name: api
          image: registry.company.com/api:v2.1.0
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
          readinessProbe:
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
            ---
apiVersion: v1
kind: Service
metadata:
  name: api-service
  namespace: production
spec:
  type: ClusterIP
  selector:
    app: api-service
  ports:
    - port: 80
      targetPort: 8080
```

3. Package Management with Helm 3

<code>helm repo add bitnami https://charts.bitnami.com/bitnami</code>	Add remote Helm chart repository index.
<code>helm repo update</code>	Fetch latest version metadata from all configured Helm repositories.
<code>helm install my-release bitnami/redis -f values-prod.yaml -n prod --create-namespace</code>	Deploy chart release with custom values override file in target namespace.
<code>helm upgrade --install my-release bitnami/redis -f values-prod.yaml -n prod</code>	Idempotent upgrade or install of chart release.
<code>helm history my-release -n prod</code>	List all past release revisions, timestamps, and deployment statuses.
<code>helm rollback my-release 2 -n prod</code>	Rollback release to previous revision 2 immediately.
<code>helm list -A</code>	List all deployed releases across all Kubernetes namespaces.

Module 7: Infrastructure as Code (IaC) with Terraform

Infrastructure as Code treats server and cloud provisioning exactly like software code: versioned, tested, repeatable, and automated.

1. Terraform CLI Workflow

<code>terraform init</code>	Initialize provider plugins, download modules, and configure remote backend.
<code>terraform fmt -recursive</code>	Automatically format all HCL configuration files to standard styling.
<code>terraform validate</code>	Validate internal syntactic correctness and variable references.
<code>terraform plan -out=tfplan.binary</code>	Generate speculative execution plan and save immutable binary artifact.
<code>terraform apply tfplan.binary</code>	Apply exact changes recorded in the saved plan binary file.
<code>terraform destroy -target=aws_instance.bastion</code>	Targeted destruction of a single resource without affecting entire stack.
<code>terraform state list</code>	List all resources currently tracked in the remote state file.
<code>terraform state show aws_s3_bucket.data_lake</code>	Inspect full metadata attributes of a specific tracked state resource.
<code>terraform import aws_vpc.main vpc-0a1b2c3d4e5f6g7h8</code>	Import existing unmanaged cloud resource into Terraform state.
<code>terraform workspace new production / select prod</code>	Manage distinct isolated environment state files with workspaces.

2. Modular Production Terraform Code (main.tf)

```
terraform {
  required_version = ">= 1.8.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.50"
    }
  }
  backend "s3" {
    bucket = "company-tf-state-prod"
    key = "vpc/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "terraform-state-lock"
    encrypt = true
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true
  tags = { Name = "${var.environment}-vpc" }
}

resource "aws_subnet" "public" {
  count = length(var.public_subnet_cidrs)
  vpc_id = aws_vpc.main.id
  cidr_block = var.public_subnet_cidrs[count.index]
  map_public_ip_on_launch = true
  tags = { Name = "${var.environment}-public-subnet-${count.index + 1}" }
}
```

Module 8: Configuration Management with Ansible

Ansible provides agentless, idempotent configuration management and application deployment over standard SSH connections.

1. Ansible Ad-Hoc Commands & Inventory Management

<code>ansible all -i inventory.ini -m ping</code>	Test SSH reachability and Python interpreter on all inventory hosts.
<code>ansible webserver -i inventory.ini -m setup -a 'filter=ansible_memtotal_mb'</code>	Gather system facts (e.g., total memory) from target host group.
<code>ansible db -i inventory.ini -m command -a 'uptime'</code>	Execute arbitrary shell command across all database nodes.
<code>ansible-playbook -i inventory.ini site.yml --syntax-check</code>	Validate YAML syntax of playbook and referenced roles.
<code>ansible-playbook -i inventory.ini site.yml --check --diff</code>	Dry-run execution displaying exact line-by-line configuration diffs.
<code>ansible-playbook -i inventory.ini site.yml -l webserver -t 'nginx'</code>	Execute only tasks tagged with 'nginx' on 'webserver' host group.
<code>ansible-vault encrypt_string 'db_password' --name 'db_pass'</code>	Encrypt inline sensitive strings using AES256 vault password.

2. Production Server Provisioning Playbook (site.yml)

```
---
- name: Provision Hardened Web Servers
  hosts: webserver
  become: yes
  vars:
    app_user: deployer

  tasks:
    - name: Update apt cache and install packages
      apt:
        name: [nginx, ufw, curl, git, htop]
        state: present
        update_cache: yes

    - name: Ensure deployment user exists
      user:
        name: "{{ app_user }}"
        shell: /bin/bash
        groups: sudo
        append: yes

    - name: Deploy custom Nginx virtualhost configuration
      template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/sites-available/default
        notify: Restart Nginx

    - name: Configure UFW firewall rules
      ufw:
        rule: allow
        port: "{{ item }}"
        proto: tcp
        loop: [22, 80, 443]

  handlers:
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
        enabled: yes
```

Module 9: CI/CD Pipelines (GitHub Actions & Jenkins)

Continuous Integration and Continuous Delivery automate code validation, container compilation, security scanning, and multi-environment deployment.

1. Enterprise GitHub Actions Pipeline (.github/workflows/deploy.yml)

```
name: Build, Test & Deploy Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
    with:
      node-version: '20'
      cache: 'npm'
      - run: |
        npm ci
        npm run lint
        npm test -- --coverage

  build-and-push:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
    with:
      registry: ${ env.REGISTRY }
      username: ${ github.actor }
      password: ${ secrets.GITHUB_TOKEN }
      - uses: docker/build-push-action@v5
    with:
      context: .
      push: true
      tags: ${ env.REGISTRY }/${ env.IMAGE_NAME }:${ github.sha },${ env.REGISTRY }/${ env.IMAGE_NAME }:latest

  deploy:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - uses: azure/k8s-set-context@v3
    with:
      kubeconfig: ${ secrets.KUBECONFIG_PROD }
      - run: |
        kubectl set image deployment/web-app web-app=${ env.REGISTRY }/${ env.IMAGE_NAME }:${ github.sha }
        -n production
        kubectl rollout status deployment/web-app -n production --timeout=180s
```

Module 10: Observability, SRE & Production Debugging

Observability is understanding internal system state through external metrics, traces, and structured logs.

1. Prometheus Configuration & PromQL Metrics Queries

<code>100 - (avg by (instance) (rate(node_cpu_seconds_total{mode='idle'}[5m])) * 100)</code>	Real-time CPU utilization % per server instance.
<code>(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / node_memory_MemTotal_bytes * 100</code>	Current RAM memory utilization %.
<code>sum(rate(http_requests_total{status=~'5..'}[5m])) / sum(rate(http_requests_total[5m])) * 100</code>	HTTP 5xx Server Error Rate percentage (Golden Signal).
<code>histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))</code>	95th percentile (p95) API response latency in seconds.
<code>sum(container_memory_working_set_bytes{namespace='production'}) by (pod)</code>	Resident memory consumed per Kubernetes pod container.

2. Advanced Linux Production Troubleshooting Toolkit

<code>lsof -i :8080</code>	Identify process name, user, and PID holding port 8080 open.
<code>lsof -p</code>	List every file descriptor, network socket, and library opened by process.
<code>strace -f -p -e trace=network,file -s 256</code>	Attach to running process to trace syscalls (file I/O, socket connections).
<code>dmesg -T grep -E 'oom-killer Out of memory'</code>	Inspect kernel ring buffer for Out-Of-Memory killed processes with human timestamps.
<code>iotop -oP</code>	Real-time disk I/O monitoring showing only processes actively performing reads/writes.
<code>nethogs eth0</code>	Display real-time network bandwidth consumption grouped by process PID.

3. Prometheus Scrape Configuration (prometheus.yml)

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'node_exporter'
    static_configs:
      - targets: ['10.0.0.10:9100', '10.0.0.11:9100']

  - job_name: 'kubernetes-pods'
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: true
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_port]
        action: replace
        target_label: __address__
        regex: (.+)
        replacement: ${1}
```

Module 11: Enterprise Capstone Project & Production Checklist

Integrate every component into a unified, enterprise-ready automated deployment architecture.

1. End-to-End Enterprise DevOps Pipeline Flow

```
[ Developer Commit ] --> [ GitHub Repo ]
|
v
[ GitHub Actions CI Pipeline ]
1. Lint & Static Analysis (SonarQube)
2. Unit & Integration Tests
3. Docker Multi-Stage Build & Security Scan (Trivy)
4. Push Signed Image to Container Registry
|
v
[ GitOps / CD Deployment ]
1. Terraform Provisions AWS EKS & RDS Infrastructure
2. ArgoCD / Helm Synchronizes Desired K8s Manifests
3. Rolling Update with Health Probes & Zero Downtime
|
v
[ SRE Observability & Alerting ]
1. Prometheus Scrapes Metrics & Node Health
2. Grafana Dashboards Display SLOs & Golden Signals
3. Alertmanager Dispatches Alerts to PagerDuty / Slack
```

2. Production Readiness Review (PRR) Checklist

Security & IAM	Enforce Least Privilege IAM roles; zero hardcoded secrets; use Vault or SealedSecrets; encrypt disks & data in transit.
High Availability	Multi-AZ deployment; minimum 3 replicas per service; PodDisruptionBudgets configured; auto-scaling enabled.
Observability	Liveness & readiness probes configured; Prometheus metrics exposed; centralized structured JSON logs enabled.
Disaster Recovery	Automated daily snapshot backups tested for point-in-time recovery; multi-region backup replication.
Capacity Planning	CPU/Memory resource requests and limits set on every container; HorizontalPodAutoscaler (HPA) active.

3. 12-Week Progressive Learning Roadmap

Weeks 1 - 2	Linux Administration, Bash Scripting, File Permissions, Text Processing & Networking Tools.
Weeks 3 - 4	Git Workflows, Branching Strategies, Interactive Rebasing & Python Automation Scripts.
Weeks 5 - 6	Docker Containerization, Multi-stage Builds, Layer Optimization & Docker Compose Stacks.
Weeks 7 - 8	Kubernetes Core Architecture, Pods, Deployments, Services, Ingress, Helm & K8s Debugging.
Weeks 9 - 10	Infrastructure as Code with Terraform Modules & Configuration Management with Ansible.
Weeks 11 - 12	CI/CD Pipelines (GitHub Actions), Prometheus & Grafana Monitoring, SRE Practices & Capstone.